C General metrics **130**
C.1 Accuracy . . . 130
C.2 Calibration and uncertainty . . . 131
C.3 Robustness . . . 133
C.4 Fairness . . . 134
C.5 Bias and stereotypes . . . 134
C.6 Toxicity . . . 137
C.7 Efficiency . . . 137

D Perturbations **139**
D.1 Robustness . . . 139
D.2 Fairness . . . 140

E Targeted evaluations **141**
E.1 Language . . . 141
E.2 Knowledge . . . 144
E.3 Reasoning . . . 144
E.4 Memorization & copyright . . . 152
E.5 Disinformation . . . 153
E.6 Bias . . . 153
E.7 Toxicity . . . 154

F Comparison with other evaluations **155**

G Contamination **155**

H Priority system **156**

I Models **158**

J Adaptation **160**
J.1 Formatting test instances . . . 160
J.2 Formatting the remainder of the prompt . . . 161
J.3 Decoding parameters . . . 161
J.4 Adaptation methods . . . 162

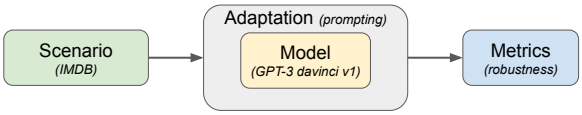

Figure 5: **Evaluation components.** Each evaluation run requires the specification of a *scenario* (what we want), a *model* with an *adaptation* process (how we get it), and one or more *metrics* (how good are the results).

Scenario: MMLU(subject=anatomy)

Input: *Which of the following terms describes the body's ability to maintain its normal state?*

References:
- *Anabolism*
- *Catabolism*
- *Tolerance*
- *Homeostasis* [correct]

Figure 6: **Scenario.** An example of a multiple choice scenario from **MMLU** (subject=anatomy), which consists of a list of instances, each with an input and a set of references.

2 Preliminaries

We introduce the basic primitives (scenario, adaptation, metric) required to evaluate a language model (Figure 5). With these primitives, we then provide a roadmap for how we holistically evaluate language models.

2.1 Scenarios

A scenario instantiates a desired use case for a language model. Useful language models are performant on a variety of scenarios: scenarios are *what* we want models *to do*. While practical use cases for language models involve other factors, we operationalize scenarios through a list of *instances*, divided into a *training* set and one or more *test* sets. Each instance consists of (i) an *input* (a string) and (ii) a list of *references*. Each reference is a string annotated with properties relevant for evaluation (e.g. is it correct or acceptable?). See Figure 6 for an example scenario.

2.2 Adaptation

Adaptation is the procedure that transforms a language model, along with training instances, into a system that can make predictions on new instances. Examples of adaptation procedures include prompting, lightweight-finetuning, and finetuning; we focus on *prompting* in this work.

We define a language model to be a black box that takes as input a *prompt* (string), along with *decoding parameters* (e.g. temperature). The model outputs a *completion* (string), along with log probabilities of the prompt and completion. We do not assume access to the internal model activations or its training data, which reflects the practical reality of API access available to researchers (Liang et al., 2022). In fact, we do not even make any assumptions about how the language model is constructed. See Figure 7 for how we adapt the example scenario from Figure 6.

Viewing language models as text-to-text abstractions is important for two reasons: First, while the prototypical LM is currently a dense Transformer trained on raw text, LMs could also use an external document store (Lewis et al., 2020c), issue search queries on the web (Nakano et al., 2021), or be trained on human preferences (Ouyang et al., 2022; Bai et al., 2022). We wish to remain agnostic to these implementation details.

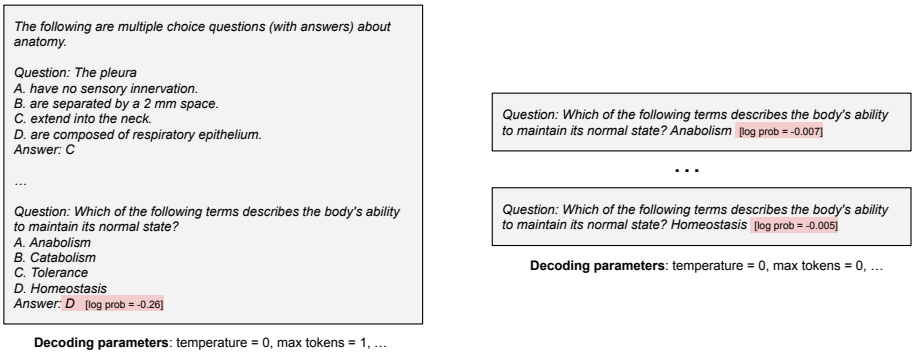

Figure 7: **Adaptation.** During adaptation, we construct a *prompt* for each evaluation instance which may include in-context training instances as well. Given *decoding parameters*, a language model generates a *completion* (in red). The multiple choice example is shown using two different adaptation strategies that we describe subsequently, with *left* version being the *joint* strategy (all answer choices are presented at once) and the *right* version being the *separate* strategy (each answer choice is presented separately).

Second, the text-to-text abstraction is a convenient general interface that can capture all the (text-only) tasks of interest, an idea that was pioneered by McCann et al. (2018) and Raffel et al. (2019).

2.3 Metrics

Once a language model is adapted, we execute the resulting system on the evaluation instances for each scenario, yielding completions with their log probabilities. To determine *how well* the model performs, we compute metrics over these completions and probabilities. Metrics concretely operationalize the abstract desiderata we require of useful systems. See §4: METRICS for more details. The metrics we compute for our running example (Figure 6) might look like this:

Exact match	:	0.571
ECE (10-bin)	:	0.221
Exact match (robustness)	:	0.551
Exact match (fairness)	:	0.524
Inference runtime	:	0.147
...		

2.4 Roadmap

To evaluate a language model, we must specify a series of runs, where each run is defined by a (scenario, adaptation method, metric) triple. Each of these scenarios, adaptation, and metrics define a complicated and structured space, which one implicitly navigates to make decisions in evaluating a language model. Central to our approach to holistic evaluation is that we make both the space and the decision explicit. In §3: CORE-SCENARIOS and §4: METRICS, we first taxonomize both spaces and then systematically select points from the spaces. This specifies our abstract aspiration and our concrete implementation, which together define HELM. Distinguishing these steps also helps clarify what is fundamentally possible vs. what we, as a specific collective of benchmark designers, chose to prioritize and emphasize. Then, we evaluate 30 models by making a specific choice for adaptation procedure (i.e. 5-shot prompting), though we emphasize many other adaptation procedures could be considered.

3 Core scenarios

We taxonomize scenarios as shown in Figure 8 based on (i) a *task* (e.g. question answering, summarization), which characterizes what we want a system to do; (ii) a *domain* (e.g. a Wikipedia 2018 dump), which